

Week 5:payment transaction management system

Scenario:

The e-commerce company needs a Payment Transaction Management System to store customer payment details, monitor transaction status, manage successful and failed payments, and generate payment reports by connecting the Payment module with the existing Customer and Order modules.

Create Payment Table:

```
SQL> CREATE TABLE Payment (Payment_ID NUMBER(7) PRIMARY KEY, Order_ID NUMBER(38), Payment_Date DATE, Payment_Mode VARCHAR2(20), Payment_Status VARCHAR2(10), Transaction_Amount NUMBER(10,2));
```

Table created.

Insert values:

```
SQL> INSERT INTO Payment VALUES (1, 1001, DATE '2026-09-27', 'UPI', 'Success', 67500);
1 row created.

SQL> INSERT INTO Payment VALUES (2, 1002, DATE '2026-09-27', 'Card', 'Success', 25000);
1 row created.

SQL> INSERT INTO Payment VALUES (3, 1003, DATE '2026-09-28', 'Cash', 'Success', 1500);
1 row created.

SQL> INSERT INTO Payment VALUES (4, 1004, DATE '2026-09-28', 'Net Banking', 'Pending', 4200);
1 row created.

SQL> INSERT INTO Payment VALUES (5, 1005, DATE '2026-09-29', 'UPI', 'Success', 8000);
1 row created.

SQL> INSERT INTO Payment VALUES (6, 1001, DATE '2026-09-29', 'Card', 'Failed', 1000);
1 row created.

SQL> INSERT INTO Payment VALUES (7, 1002, DATE '2026-09-30', 'UPI', 'Pending', 5000);
1 row created.

SQL> INSERT INTO Payment VALUES (8, 1003, DATE '2026-09-30', 'Net Banking', 'Failed', 2000);
1 row created.
```

Store payment details:

```
SQL> SELECT * FROM Payment;
```

PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST
1	1001	27-SEP-26	UPI	Success
2	1002	27-SEP-26	Card	Success
3	1003	28-SEP-26	Cash	Success
4	1004	28-SEP-26	Net Banking	Pending
5	1005	29-SEP-26	UPI	Success
6	1001	29-SEP-26	Card	Failed
7	1002	30-SEP-26	UPI	Pending
8	1003	30-SEP-26	Net Banking	Failed

8 rows selected.

Constraints to the Payment Table

Foreign Key Constraint:

```
SQL> ALTER TABLE Payment ADD CONSTRAINT FK_Payment_Order FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID);
```

Table altered.

NOT NULL Constraint on Payment_Mode and Payment_Status:

```
SQL> ALTER TABLE Payment MODIFY Payment_Mode NOT NULL;
```

Table altered.

```
SQL> ALTER TABLE Payment MODIFY Payment_Status NOT NULL;
```

Table altered.

CHECK Constraint on Transaction_Amount:

```
SQL> ALTER TABLE Payment ADD CONSTRAINT CHK_Transaction_Amount CHECK (Transaction_Amount > 0);  
Table altered.
```

Report 1: Payment Mode Analysis

Find 1: The number of transactions using UPI.

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS FROM Payment WHERE Payment_Mode = 'UPI';  
  
UPI_TRANSACTIONS  
-----  
3
```

Find 2: The number of card payments.

```
SQL> SELECT COUNT(*) AS CARD_PAYMENTS FROM Payment WHERE Payment_Mode IN ('Credit Card', 'Debit Card', 'Card');  
  
CARD_PAYMENTS  
-----  
2
```

Find 3: The most preferred payment method.

```
SQL> SELECT Payment_Mode, COUNT(*) AS TRANSACTION_COUNT FROM Payment GROUP BY Payment_Mode ORDER BY TRANSACTION_C  
OUNT DESC FETCH FIRST 1 ROW ONLY;  
  
PAYMENT_MODE      TRANSACTION_COUNT  
-----  
UPI                3
```

Report 2: Revenue Analysis

Find 1: The total revenue generated from successful payments.

```
SQL> SELECT SUM(Transaction_Amount) AS Total_Revenue FROM Payment WHERE Payment_Status = 'Success';  
  
TOTAL_REVENUE  
-----  
102000
```

Find 2: The revenue generated by each payment method from successful payments.

```
SQL> SELECT Payment_Mode, SUM(Transaction_Amount) AS Total_Revenue FROM Payment WHERE Payment_Status = 'Success' GROUP BY Payment_Mode ORDER BY Total_Revenue DESC;
```

PAYMENT_MODE	TOTAL_REVENUE
UPI	75500
Card	25000
Cash	1500

Find 3: The average transaction amount for successful payments.

```
SQL> SELECT ROUND(AVG(Transaction_Amount), 2) AS Average_Transaction_Amount FROM Payment WHERE Payment_Status = 'Success';
```

AVERAGE_TRANSACTION_AMOUNT
25500

Report 3: Customer Payment History

Display:

- Customer name
- Order ID
- Payment mode
- Amount
- Payment status

```
SQL> SELECT C.Customer_Name, O.Order_ID, P.Payment_Mode, P.Transaction_Amount AS Amount, P.Payment_Status FROM Customer C JOIN Orders O ON C.Customer_ID = O.Customer_ID JOIN Payment P ON O.Order_ID = P.Order_ID ORDER BY P.Payment_Date;
```

CUSTOMER_NAME	ORDER_ID
Arun	1001
UPI	67500 Success
Priya	1002
Card	25000 Success
Kumar	1003
Cash	1500 Success

CUSTOMER_NAME	ORDER_ID
Divya	1004
Net Banking	4200 Pending
Rahul	1005
UPI	8000 Success
Arun	1001
Card	1000 Failed

CUSTOMER_NAME			ORDER_ID
PAYMENT_MODE	AMOUNT	PAYMENT_ST	
Priya			1002
UPI	5000	Pending	
Kumar			1003
Net Banking	2000	Failed	

8 rows selected.

Payment analysis:

```
SQL> SELECT Payment_Mode, COUNT(*) AS Transaction_Count, SUM(Transaction_Amount) AS Total_Revenue, ROUND(AVG(Transaction_Amount), 2) AS Avg_Transaction FROM Payment WHERE Payment_Status = 'Success' GROUP BY Payment_Mode ORDER BY Total_Revenue DESC;
```

PAYMENT_MODE	TRANSACTION_COUNT	TOTAL_REVENUE	AVG_TRANSACTION
UPI	2	75500	37750
Card	1	25000	25000
Cash	1	1500	1500

Result

Thus, the Payment Transaction Management System was successfully implemented using SQL. The Payment table was created and linked with the Orders table using a foreign key. Payment analysis, revenue analysis, and customer payment history reports were generated successfully.